Graphs — Cheat Sheet

Personalized from your dsa-tutor progress in `graphs/` · generated 2026-08-15

● solid — you've got this ● shaky / learning — watch the flagged trap ● not yet studied — reference only

● Terminology & Representations SHAKY

Google Maps: intersections = vertices, roads = edges. One-way streets = directed. Distances = weights.

List = Lean (sparse), **Matrix = Massive** (dense). Matrix size = V² *always*, edges never shrink it.
THE CURRENCY: List pays per **FRIEND** (degree). Matrix pays per **SEAT** in the row (V). Matrix's one superpower: edge u–v? in O(1).

Watch: a 10M-node matrix is 100 trillion cells even with zero edges — size is never edge-count. And list neighbor-iteration is O(degree), not O(V) — don't price it like the matrix.

Operation	Adj. List	Adj. Matrix
Add edge	O(1)	O(1)
Edge u–v exists?	O(deg(u))	O(1)
Iterate neighbors of u	O(deg(u))	O(V)
Space	O(V+E)	O(V²)

● Breadth-First Search SOLID

A ripple spreading from a dropped stone: reaches everything 1 hop away, then 2 hops, never skipping ahead.

Mark visited on ENQUEUE, not dequeue — else the same node enters the queue twice. FIFO keeps the wave **SORTED**: the queue only ever holds ring d and ring d+1, so first-arrival = shortest.

```
from collections import deque
def bfs(graph, start):
    visited = {start}
    q = deque([start])
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)     # mark on enqueue
                q.append(nb)
    return order
```

Use: shortest path / min hops on *unweighted* graphs. **Time** O(V+E), **space** O(V).

● Depth-First Search SOLID

Maze walk: take the first unexplored corridor, dive until a dead end, backtrack to the last junction with an unexplored path.

Depth = Stack (LIFO) — the call stack *is* the stack in recursion. **Mark BEFORE you recurse**, never after (else infinite recursion on a revisit).

```
def dfs(graph, start, visited=None, order=None):
    if visited is None: visited, order = {start}, [start]
    for nb in graph[start]:
        if nb not in visited:
            visited.add(nb)          # mark before recurs
            order.append(nb)
            dfs(graph, nb, visited, order)
    return order
```

Iterative: explicit stack, mark-on-push avoids stack duplicates (mark-on-pop + skip-guard also valid, mirrors recursion). Recursion dies past Python's ~1000-frame limit; iterative doesn't. **Time** O(V+E), **space** O(V).

● Traversal Patterns SHAKY

*ONE engine, TWO knobs. BFS/DFS never change — only the **neighbor function** and whether you need a **visited** set.*

Structure	Neighbors	Visited
Tree	children	not needed
Graph	adj[node]	set
Grid	4 dirs + bounds	mark cell in place
Matrix	j where M[i][j]==1, j≠i	set

Shortest / levels → **BFS**. Explore-all / components / cycle / paths → **DFS**. Self-loop lives on the **DIAGONAL** M[i][i].

Watch (index-vs-value): `[j for j, val in enumerate(M[i]) if val==1 and j!=i]` — enumerate yields (index, value) *index-first*. The index is the neighbor; the value is only the 0/1 porch light. If the thing you're indexing with can only be 0 or 1, you're using a light switch as an address.

Connected Components

SOLID

Islands in an ocean: each maximal group of mutually-reachable nodes is one island.

Outer loop = find dry land; inner traversal = flood the whole island. `count++` per fresh island.

```
def count_components(graph):
    visited, count = set(), 0
    for node in graph:
        if node not in visited:
            count += 1
            # bfs/dfs flood, marking visited
    return count
```

Via DSU — count the retired leaders: every successful `union` fuses two groups into one (fuses exactly one leader out). `components = n - sum(uf.union(u,v) for u,v in edges)`. find-only diagnoses without treating — you must actually call union.

Cycle Detection

SOLID

*A cycle == a **BACK EDGE**, nothing else. Plain visited (1 axis) only guarantees termination; cycle detection needs a 2nd axis — still on my current path?*

Directed (3 colors — WHITE/GRAY/BLACK): cycle ⟺ edge to a **GRAY** node (on the DFS stack). Edge to BLACK = harmless convergence. *Gray = on the stove; point back to gray and the snake eats its own tail.*

Undirected: no cross/forward edges exist — a visited non-parent neighbor is *always* a back edge, so plain visited + parent check suffices. **Parent-skip is a feature here, a bug in directed.**

Watch (U-turn vs roundabout): one undirected edge walked back (A–B) is a U-turn, not a cycle (parent-skip exists precisely for this). Directed A→B, B→A are two *distinct* edges = a real 2-cycle — must NOT skip the parent there.

```
def has_cycle_directed(graph):
    NOT_EXP, ON_STACK, DONE = 0, 1, 2
    status = defaultdict(int)
    def dfs(node):
        status[node] = ON_STACK
        for nb in graph[node]:
            if status[nb] == ON_STACK: return True
            if status[nb] == NOT_EXP and dfs(nb): return
        status[node] = DONE
        return False
    return any(status[n]==NOT_EXP and dfs(n) for n in gra

def has_cycle_undirected(graph):
    visited = set()
    def dfs(node, parent):
        visited.add(node)
        for nb in graph[node]:
            if nb == parent: continue
            if nb in visited or dfs(nb, node): return Tru
        return False
    return any(n not in visited and dfs(n, -1) for n in g
```

Topological Sort — Kahn's

LEARNING

*A **LINEUP**, not a walk: lay all nodes on a line; valid iff every edge points **RIGHT**. Consecutive nodes need not be adjacent. Getting dressed: socks before shoes.*

In-degree = your **WAIT COUNT**; zero = free to go. **SHORT LINEUP = CYCLE** — if `len(order) < n` when the pool runs dry, the leftovers are all waiting on each other.

Watch: a topo order is not a path — A,B,C,D can be valid for A→C, B→C, C→D even though A→B isn't an edge. Also: check the cycle rule by counting the *lineup*, not whether the pool ever went empty (it can start non-empty and still deadlock partway).

```
def topo_sort(graph):
    indeg = {u: 0 for u in graph}
    for u in graph:
        for v in graph[u]: indeg[v] += 1
    q = deque([u for u in indeg if indeg[u]==0])
    order = []
    while q:
        u = q.popleft(); order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return order if len(order)==len(graph) else None  # N
```

Alt: DFS post-order — push a node once fully DONE, reverse the stack at the end. (Same 3-color machinery as cycle detection.)

Union-Find (DSU)

LEARNING

***GROUPS and LEADERS.** Each member stores only its immediate boss (parent pointer). `find` = climb the boss chain. `union` = the losing leader gets a new boss.*

THE LOSER PAYS — small joins large, so depth grows only by losing (losing doubles your group ⟹ height ≤ $\log_2 n$). **Path compression**: climb once, everyone met gets the leader's speed dial. Together: ~O(1) amortized (inverse Ackermann). **Union links the two LEADERS**, never the raw edge endpoints.

Watch: union-by-size must absorb the loser's *whole roster* — `size[leader] += size[loser]`, not `+= 1` (the `+= 1` bug hides whenever the loser happens to be a singleton).

```
class UnionFind:
    def __init__(self): self.leader, self.size = {}, {}
    def find(self, x):
        if x != self.leader.setdefault(x, x):
            self.leader[x] = self.find(self.leader[x])
        return self.leader[x]
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False        # already same g
        sa, sb = self.size.setdefault(ra,1), self.size.se
        if sa < sb: ra, rb = rb, ra
        self.leader[rb] = ra; self.size[ra] += self.size[
        return True
```

Use: cycle test on undirected edges, Kruskal's MST, counting components.

Bipartite Check

NOT STARTED

2-coloring: can every node be RED/BLUE so no edge joins same colors? Equivalent to "no odd-length cycle."

```
def is_bipartite(graph):
    color = {}
    for start in graph:
        if start in color: continue
        color[start] = 0
        q = deque([start])
        while q:
            u = q.popleft()
            for v in graph[u]:
                if v not in color:
                    color[v] = 1 - color[u]; q.append(v)
                elif color[v] == color[u]:
                    return False
    return True
```

Neighbor gets the opposite color of current; same color on both ends of an edge ⟹ not bipartite. **Time** O(V+E).

Shortest Paths

NOT STARTED

Unweighted → BFS. First-arrival = shortest (ring invariant, see BFS card).

Dijkstra's (weights ≥ 0): BFS with a min-heap instead of FIFO — always expand the closest unvisited node. `dist[start]=0`; pop smallest; relax: `dist[v]=min(dist[v], dist[u]+w)`. Breaks with negative edges (greedy commit can be undercut later). **O((V+E) log V)**.

Bellman-Ford (handles negative weights): relax *every* edge, V−1 times — brute-force patience instead of a priority queue. One extra pass: if any distance still improves ⟹ negative cycle. **O(V·E)**.

Floyd-Warshall (all-pairs): try every node k as a waypoint between every pair (i,j): `dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])`. **O(V³)**.

Minimum Spanning Tree

NOT STARTED

Connect all V nodes with V−1 edges, minimum total weight, no cycles.

Kruskal's: sort all edges ascending by weight; greedily add an edge if it doesn't close a cycle — literally the **Union-Find** redundant-edge check, run over sorted edges. **O(E log E)**.

Prim's: grow one tree from a start node; repeatedly pull the cheapest edge connecting a new node (min-heap of frontier edges) — like Dijkstra's but the priority is edge weight *to the tree*, not cumulative distance from source. **O(E log V)**.

Complexity Summary

Algorithm	Time	Space	Notes
BFS / DFS	O(V+E)	O(V)	traversal engine underlying most of the below
Cycle detection	O(V+E)	O(V)	DFS + color / parent-check
Topological sort (Kahn's)	O(V+E)	O(V)	fails ⟺ cycle exists
Union-Find (find/union)	~O(α(n)) amortized	O(V)	path compression + union by size
Bipartite check	O(V+E)	O(V)	BFS/DFS 2-coloring
Dijkstra's (binary heap)	O((V+E) log V)	O(V)	no negative weights
Bellman-Ford	O(V·E)	O(V)	negative weights OK; detects negative cycles
Floyd-Warshall	$O(V^3)$	$O(V^2)$	all-pairs shortest path
Kruskal's MST	O(E log E)	O(V)	sort + Union-Find
Prim's MST	O(E log V)	O(V)	min-heap frontier, Dijkstra-shaped